ORIENTAÇÕES

Entrega de Trabalho

1. TÍTULO:

KernelLab — Simulador de Gerenciamento de CPU, Memória e Dispositivos em Sistemas Multitarefa

2. DESAFIO:

A **MedControl Systems** desenvolve uma plataforma utilizada para monitorar equipamentos médicos conectados. Seu servidor executa simultaneamente processos de monitoramento, emissão de alarmes, transmissão de dados, geração de relatórios, backup e atualização do sistema.

Nas últimas semanas, foram identificados:

- atraso no processamento de alarmes;
- aumento do tempo de resposta;
- disputa pela CPU;
- elevado número de faltas de página;
- processos de baixa prioridade aguardando por períodos excessivos;
- concorrência no acesso aos dispositivos;
- risco de processos não autorizados utilizarem recursos críticos.

Você foi contratado(a) como **Engenheiro(a) de Computação** para desenvolver o **KernelLab**, um simulador de gerenciamento de recursos capaz de comparar diferentes políticas de escalonamento, substituição de páginas e controle de acesso a dispositivos.

ORIENTAÇÕES

Entrega de Trabalho

O objetivo não é desenvolver um sistema operacional completo. Você deverá construir uma aplicação que reproduza as decisões tomadas por um sistema operacional e permita avaliar o impacto de cada algoritmo.

O simulador deverá seguir o fluxo:

Entrada dos processos → escalonamento da CPU → cálculo das métricas → gerenciamento da memória → controle dos dispositivos → comparação dos resultados

Você poderá implementar o projeto em **Python**, linguagem recomendada para a atividade, ou utilizar C, C++ ou Java mediante justificativa técnica.

Dados dos processos

Considere um sistema com apenas uma CPU e utilize os seguintes processos:

Processo	Função	Chegada	Burst de CPU	Prioridade	Memória	Dispositivo
P1	Monitoramento cardíaco	0	8	1	120 MB	Rede
P2	Transmissão de telemetria	1	4	2	80 MB	Rede
P3	Geração de relatórios	2	9	4	200 MB	Disco
P4	Emissão de alarme crítico	3	2	0	60 MB	Áudio e rede
P5	Backup do sistema	5	12	5	300 MB	Disco
P6	Atualização de dados	6	6	3	150 MB	Disco e rede

ORIENTAÇÕES

Entrega de Trabalho

Considere que o menor número representa a maior prioridade.

Etapa 1 — Escalonamento da CPU

Implemente obrigatoriamente:

- FCFS — First Come, First Served;
- SJF — Shortest Job First;
- Round Robin com quantum igual a 3;
- escalonamento por prioridade.

Para cada algoritmo, apresente:

- ordem de execução;
- Diagrama de Gantt;
- tempo de espera;
- tempo de retorno;
- tempo de resposta;
- média de cada métrica;
- quantidade de trocas de contexto;
- processos sujeitos a atraso excessivo ou starvation.

O simulador deverá permitir alterar os dados dos processos e o quantum do Round Robin sem modificar diretamente a lógica dos algoritmos.

Etapa 2 — Comparação dos escalonadores

Compare os algoritmos e responda:

- Qual apresentou o menor tempo médio de espera?

- Qual ofereceu melhor tempo de resposta?
- Qual tratou o processo de alarme com maior eficiência?
- Qual apresentou maior risco de starvation?
- Qual distribuiu a CPU de maneira mais justa?
- Qual seria mais adequado para o cenário da MedControl?

Sua escolha deverá considerar desempenho, prioridade, justiça e criticidade, e não apenas uma única métrica.

Etapa 3 — Gerenciamento de memória

Considere uma memória física com **quatro frames** e a seguinte sequência de referências:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5, 2, 1, 5, 3, 2

Implemente os algoritmos:

- FIFO;
- LRU;
- Optimal, utilizado como referência comparativa.

Para cada algoritmo, apresente:

- estado dos frames após cada referência;
- quantidade de page hits;
- quantidade de page faults;
- taxa de faltas de página;
- algoritmo com melhor resultado;
- justificativa da diferença encontrada.

O simulador deverá permitir alterar a sequência de páginas e a quantidade de frames.

Etapa 4 — Controle de dispositivos

Modele o acesso aos dispositivos compartilhados:

- disco;
- rede;
- sistema de áudio;
- arquivos de log.

Crie uma matriz de controle de acesso indicando quais processos poderão:

- ler;
- gravar;
- executar;
- utilizar o dispositivo;
- solicitar acesso exclusivo.

Implemente uma fila simples para representar solicitações concorrentes a pelo menos um dispositivo. Utilize mutex, semáforo ou mecanismo equivalente para demonstrar como o sistema evita acessos simultâneos indevidos.

Etapa 5 — Resultados e visualização

Apresente os resultados em tabelas e gráficos contendo:

- comparação dos tempos médios;
- trocas de contexto;
- page faults;

- page hits;
- taxa de faltas;
- desempenho dos algoritmos;
- análise de justiça;
- recomendação final.

A interface poderá ser desenvolvida em terminal, desktop ou web. O funcionamento dos algoritmos e a qualidade da análise serão mais importantes que a sofisticação visual.

Etapa 6 — Organização do projeto

Estrutura recomendada:

```
kernellab/
├── src/
│   ├── main.py
│   ├── schedulers.py
│   ├── memory.py
│   ├── devices.py
│   └── metrics.py
├── data/
│   └── processes.json
├── tests/
├── docs/
├── requirements.txt
├── README.md
└── .gitignore
```

ORIENTAÇÕES

Entrega de Trabalho

O `README.md` deverá apresentar:

- problema e objetivo;
- arquitetura do simulador;
- algoritmos implementados;
- instruções de instalação e execução;
- formato dos dados de entrada;
- exemplos de uso;
- instruções para execução dos testes;
- resultados principais;
- limitações e possíveis melhorias.

3. FONTE DE PESQUISA:

Utilize todas as unidades, videoaulas, textos-base, atividades e materiais complementares disponibilizados na disciplina **Sistemas Operacionais** no AVA.

Livro: Sistemas Operacionais Modernos, de Andrew S. Tanenbaum e Herbert Bos
Este livro é um clássico sobre sistemas operacionais e fornece uma visão abrangente do gerenciamento de recursos. O capítulo 3, "Gerenciamento de Processos", discute os conceitos básicos de multitarefa e os diferentes mecanismos de gerenciamento de processos.
Link:
https://www.amazon.com.br/Sistemas-operacionais-modernos-Andrew-Tanenbaum-ebook/dp/B01IPIN4W4

Artigo: Organização E Arquitetura De Computadores Ii Sistemas Multiprogramados

ORIENTAÇÕES

Entrega de Trabalho

Esse artigo trata dos sistemas multiprogramados e possui uma boa base inicial de pesquisa para o tema do estudo proposto.
Link:
https://www.docentes.univasf.edu.br/max.santana/material/OAC-II/sistemas_multiprogramados.pdf

Artigo: INF 1316: Sistemas Operacionais
Esse artigo trata de sistemas operacionais variados e tem uma boa base sobre o assunto a ser discutido nesse caso de estudo.
Link:
https://www-di.inf.puc-rio.br/~endler/courses/inf1019/transp/aulas-teoricas/cap-1.pdf

Site: Tecnoblog - O que é um sistema operacional?
Essa página traz um conteúdo interessante abordando o sistema operacional e suas divisões, trazendo uma visão clara de apoio a esse caso de estudo.
Link: https://tecnoblog.net/responde/o-que-e-um-sistema-operacional/

4. ENTREGÁVEIS

Sua entrega final da disciplina **Sistemas Operacionais** será composta por **três entregáveis obrigatórios**. A ausência de qualquer um deles comprometerá a avaliação integral do projeto.

Parte Teórica — 25% da nota

Entregue um **relatório técnico em PDF** contendo:

- contextualização do problema;

- estados e ciclo de vida dos processos;
- explicação dos quatro algoritmos de escalonamento;
- conceitos e métricas de desempenho;
- funcionamento da memória virtual;
- explicação dos algoritmos FIFO, LRU e Optimal;
- conceitos de concorrência, mutex e semáforos;
- segurança no acesso aos dispositivos;
- relação entre os conceitos e as decisões implementadas;
- referências utilizadas.

Você deverá relacionar a teoria ao cenário da MedControl. A reprodução isolada de definições não será considerada uma análise suficiente.

Parte Prática — 50% da nota

Entregue um **repositório público no GitHub** contendo:

- simulador funcionando;
- FCFS, SJF, Round Robin e Prioridade;
- Diagramas de Gantt;
- cálculo das métricas;
- FIFO, LRU e Optimal;
- cálculo de page hits e page faults;
- matriz de controle de acesso;
- simulação de concorrência em dispositivo;
- tabelas ou gráficos comparativos;
- testes dos principais algoritmos;
- código organizado e documentado;

ORIENTAÇÕES

Entrega de Trabalho

- `README.md` com instruções completas.

Finalize a aplicação com uma recomendação técnica indicando qual política seria mais adequada ao cenário e quais limitações ainda permaneceriam.

O projeto deverá ser reproduzível. O avaliador precisa conseguir clonar o repositório, seguir o `README.md`, executar o simulador e validar os resultados.

Vídeo Pitch — 25% da nota

Grave um vídeo de **até 4 minutos**, apresentando:

- o problema enfrentado pela MedControl;
- os algoritmos de escalonamento;
- a execução do simulador;
- o Diagrama de Gantt;
- a comparação das métricas;
- a simulação da memória;
- os resultados dos algoritmos de substituição;
- o controle de acesso aos dispositivos;
- sua recomendação final.

Você deverá demonstrar o sistema funcionando. Não será suficiente apenas apresentar slides, exibir o código sem executá-lo ou ler o relatório.

Publique o vídeo no **YouTube, podendo configurá-lo como não listado, ou em outra plataforma que permita acesso para avaliação**.

Antes da entrega, verifique se os links do GitHub e do vídeo estão corretos, públicos e acessíveis. Inclua o link do vídeo no final do relatório e no `README.md`.

ORIENTAÇÕES

Entrega de Trabalho



Ao final, você terá um **simulador de gerenciamento de recursos de sistemas operacionais** para incluir em seu portfólio, comprovando conhecimentos em **processos, escalonamento, memória virtual, algoritmos, concorrência, métricas e controle de dispositivos**.